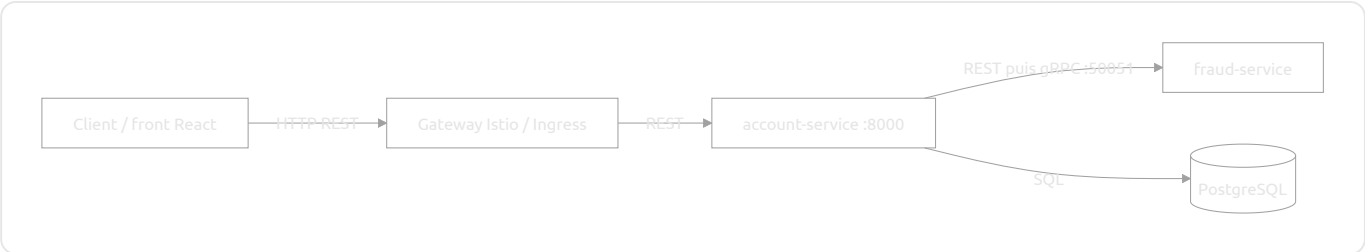
SafeBank : documentation technique

1. Stack

Domaine	Choix
Langage	Python 3.12 (le prof a autorisé Python à la place de Java/Spring)
Gestion d'environnement	**uv** (un projet uv par service : `pyproject.toml` + `uv.lock`)
API REST	FastAPI + Pydantic v2
gRPC	`grpcio` + `grpcio-tools` (code généré depuis un fichier `.proto`)
Base de données	PostgreSQL ; accès via SQLAlchemy 2 (ou SQLModel) ; migrations Alembic
Tests	pytest (+ `httpx` pour le client de test FastAPI)
Qualité	ruff (lint + format)
Conteneurs	Docker, images publiées sur Docker Hub
Orchestration	Kubernetes (minikube en local), Istio pour la gateway et le mTLS
Cloud	Google Cloud (GKE)
CI	GitHub Actions (un workflow par service)

2. Architecture



- Seul `account-service` est exposé à l'extérieur, via la gateway.
- `fraud-service` n'est **jamais** exposé à l'extérieur : il n'est joignable qu'à l'intérieur du cluster.
- `fraud-service` est **sans état** : `account-service` lui envoie toutes les informations utiles (montant, solde, nombre de virements récents, bénéficiaire connu ou non). On peut donc le dupliquer (plusieurs pods) sans problème de données.

3. Organisation du dépôt (monorepo)

```
safebank/
├── README.md                  # présentation courte + comment lancer
├── docs/                      # ces documents + captures pour le rapport
├── proto/
│   └── fraud.proto            # contrat gRPC partagé entre les deux services
├── services/
│   ├── account-service/       # service 1, totalement autonome
│   └── fraud-service/         # service 2, totalement autonome
├── k8s/                       # ressources communes du cluster
│   ├── namespace.yaml
│   ├── gateway/               # Ingress ou Istio Gateway + VirtualService
│   ├── postgres/              # StatefulSet/Deployment, PVC, Service, Secret
│   └── security/              # RBAC, PeerAuthentication (mTLS)
├── frontend/                  # option : front React
└── .github/workflows/         # CI : un workflow par service
```

Règle d'or : chaque service est autonome. Il a son propre `pyproject.toml`, son propre `uv.lock`, son propre `Dockerfile`, son propre `.dockerignore`, ses propres tests et ses propres fichiers Kubernetes. On n'utilise pas de « workspace » uv commun : cela couplerait les dépendances des services entre eux.

4. Architecture en couches (dans chaque service)

C'est le découpage vu en cours (entity / service / web), adapté à Python :

```
services/account-service/
├── pyproject.toml
├── uv.lock
├── .python-version
├── Dockerfile
├── .dockerignore
├── README.md
├── k8s/
│   ├── deployment.yaml
│   ├── service.yaml
│   └── configmap.yaml
├── app/
│   ├── main.py                # crée l'app FastAPI, branche les routeurs
│   ├── config.py              # configuration lue depuis les variables
d'environnement
│   ├── entity/                # classes métier (Account, Transfer,
TransferStatus)
│   ├── dto/                   # schémas Pydantic d'entrée / sortie de l'API
│   ├── repository/            # accès aux données (mémoire, puis PostgreSQL)
│   ├── service/               # logique métier (règles de virement, appel
anti-fraude)
│   ├── client/                # client vers fraud-service (REST, puis gRPC)
│   ├── web/                   # routes FastAPI (= contrôleurs) + dépendances
│   └── exceptions.py          # erreurs métier (AccountNotFound,
InsufficientFunds...)
└── tests/
    ├── unit/                  # tests de la couche service, sans HTTP
    └── api/                   # tests des routes avec le TestClient
```

Couche	Équivalent Spring vu en cours	Responsabilité	Ne doit PAS
`entity`	`@Entity`	Représenter les objets métier	dépendre de FastAPI
`dto`	DTO	Valider ce qui entre et sort de l'API	contenir de logique métier
`repository`	`@Repository`	Lire / écrire les données	contenir de règles métier
`service`	`@Service`	Appliquer les règles métier	connaître HTTP (pas de `HTTPException` ici)
`client`	`RestTemplate` / stub gRPC	Appeler un autre microservice	décider à la place du service
`web`	`@RestController`	Recevoir la requête HTTP, appeler le service, renvoyer la réponse	accéder directement au repository

Sens des dépendances : `web → service → repository → entity` (et `service → client`). Le service lève des exceptions métier (`InsufficientFunds`), la couche `web` les traduit en codes HTTP (`422`, `404`…). Avantage : la logique métier se teste sans lancer de serveur, et on peut changer la base de données ou le protocole (REST → gRPC) sans toucher aux règles.

`fraud-service` suit la même structure, avec en plus un dossier `app/grpc/` (serveur gRPC + code généré depuis `proto/fraud.proto`).

5. Contrats d'API

account-service (REST, port 8000)

Méthode	Route	Description	Codes
GET	`/health`	Sonde de santé (Kubernetes)	200
POST	`/accounts`	Ouvrir un compte `{owner, initial_balance}`	201, 422
GET	`/accounts`	Lister les comptes	200
GET	`/accounts/{id}`	Détail d'un compte	200, 404
POST	`/transfers`	Virement `{from_account_id, to_account_id, amount}`	201, 404, 422, 503
GET	`/accounts/{id}/transfers`	Historique des virements d'un compte	200, 404

Règles de `POST /transfers` (dans la couche `service`) :

1. les deux comptes existent (sinon 404) ;
2. `amount > 0`, deux décimales maximum (sinon 422) ;
3. compte source différent du compte destination (sinon 422) ;
4. solde suffisant (sinon 422) ;
5. appel à `fraud-service` → `ALLOW` : débit/crédit et statut `ACCEPTED` ; `REVIEW` : statut `PENDING_REVIEW` ; `BLOCK` : statut `REJECTED` ;
6. si `fraud-service` ne répond pas : **on refuse par sécurité** (503), aucun argent ne bouge.

Les montants sont toujours des `Decimal`, jamais des `float`.

fraud-service

REST (port 8001) : `POST /score`

```json
// requête
{"amount": "1200.00", "balance": "1500.00", "recent_transfers": 1, "known_benefic
// réponse
{"score": 80, "decision": "BLOCK", "reasons": ["AMOUNT_OVER_1000", "AMOUNT_OVER_8
```

gRPC (port 50051), fichier `proto/fraud.proto` :

```proto
```

```proto
syntax = "proto3";
package safebank.fraud.v1;

service FraudService {
  rpc Score (ScoreRequest) returns (ScoreResponse);
}

message ScoreRequest {
  string amount = 1;             // montant en texte pour garder la précision déci
  string balance = 2;
  int32 recent_transfers = 3;
  bool known_beneficiary = 4;
}

enum Decision {
  DECISION_UNSPECIFIED = 0;
  ALLOW = 1;
  REVIEW = 2;
  BLOCK = 3;
}

message ScoreResponse {
  int32 score = 1;
  Decision decision = 2;
  repeated string reasons = 3;
}
```

6. Configuration (variables d'environnement)

Variable	Service	Exemple
`FRAUD_SERVICE_URL`	account	`http://fraud-service:8001`
`FRAUD_GRPC_TARGET`	account	`fraud-service:50051`
`FRAUD_TRANSPORT`	account	`rest` ou `grpc`
`DATABASE_URL`	account	`postgresql+psycopg://safebank:***@postgres:5432/safebank`

Lues dans `app/config.py` (pydantic-settings). En Kubernetes : valeurs non sensibles dans un **ConfigMap**, mot de passe de la base dans un **Secret**. Jamais de mot de passe dans le code ni dans Git.

7. Docker

- Un `Dockerfile` **dans le dossier de chaque service.**
- Image basée sur `python:3.12-slim`, dépendances installées avec `uv sync --frozen --no-dev` **avant** de copier le code (cache Docker).
- L'application tourne avec un **utilisateur non root.**
- Images : `artmis137/safebank-account-service:<version>` et `artmis137/safebank-fraud-service:<version>`. On utilise des **tags versionnés** (`0.1.0`, `0.2.0`…), pas seulement `latest`.

8. Kubernetes

Par service (`services/<service>/k8s/`) :

- **Deployment** : image versionnée, 2 réplicas, `livenessProbe` et `readinessProbe` sur `/health`, limites CPU/mémoire.
- **Service** de type `ClusterIP` (l'accès extérieur passe par la gateway).
- **ConfigMap** pour la configuration.

Commun (`k8s/`) : namespace `safebank`, gateway (Ingress ou Istio Gateway + VirtualService), PostgreSQL (PVC + Secret), sécurité (RBAC, mTLS).

9. Conventions de travail

- **Branches** : `main` est protégée ; on travaille sur `feature/<sujet>` (ex. `feature/transfers`), puis Pull Request relue par l'autre membre du binôme.
- **Commits** au format Conventional Commits : `feat:`, `fix:`, `test:`, `docs:`, `build:`, `ci:`, `refactor:`.
- **Avant chaque PR** : `uv run ruff check .`, `uv run ruff format .`, `uv run pytest` doivent passer.
- **Commandes uv utiles** :
  - `uv sync` : installer l'environnement du service
  - `uv add <paquet>` / `uv add --dev <paquet>` : ajouter une dépendance
  - `uv run uvicorn app.main:app --reload` : lancer le service en local
  - `uv run pytest` : lancer les tests
- Les captures d'écran pour le rapport sont rangées au fil de l'eau dans `docs/captures/`.